End-Report: Hand-Drawn Sketch Recognition

B23CM1055	B23CM1061	B23EE1029	B23EE1054
Pragati Rokade	Vandita Gupta	Jigyasa Tiwari	Pragati Khandelwal

April 10, 2025

Abstract

Hand-drawn sketch recognition presents unique challenges due to the abstract, sparse, and highly varied nature of sketches compared to natural images. This project aims to classify human-drawn sketches into predefined object categories by leveraging a combination of traditional and deep learning-based feature extraction methods. We preprocess sketches to enhance quality and reduce noise, extract visual features using Histogram of Oriented Gradients (HOG), Local Binary Patterns (LBP), and Convolutional Neural Networks (CNNs), and apply multiple machine learning classifiers to evaluate performance.

Keywords: Sketch recognition, Feature extraction, HOG, LBP, CNN features, Machine learning, Computer vision

Contents

1 Introduction

Hand-drawn sketch recognition aims to classify sketches drawn by humans into predefined categories automatically. Unlike natural photographs, sketches are often abstract, minimal, and highly personal—making it difficult for machines to understand them in the way humans do.

This project takes inspiration from the study "How Do Humans Sketch Objects?" (Eitz et al., SIGGRAPH 2012), which explored how people sketch everyday objects and created one of the first large-scale datasets for sketch analysis laying the groundwork for computational models to learn from these patterns.

Our approach involves three key stages: (1) image preprocessing to enhance quality and reduce noise, (2) multi-modal feature extraction using HOG, LBP, and CNN-based features, and (3) classification using various machine learning algorithms. We also explore dimensionality reduction to ensure the system remains computationally efficient.

2 Approaches Tried

We initially explored basic feature extraction techniques such as Edge Detection, Corner Detection, and Local Binary Pattern (LBP). These were implemented using image processing libraries such as OpenCV and evaluated on 128x128 grayscale sketch images. These methods aimed at capturing basic structural and texture details of the sketches to support the downstream classification tasks.

2.1 Image Preprocessing

Before feature extraction, we applied several preprocessing techniques to enhance the quality of the sketch images:

2.1.1 Image Resizing and Normalization

All images were resized to 128×128 pixels and normalized to the range [0,1] to ensure uniformity across the dataset. This resizing step ensures consistency in image dimensions and reduces computational complexity. Normalization helped to bring all pixel values to the same scale, which improved the training efficiency of the classification algorithms. The standardization of input images ensures that the feature extraction process works efficiently without bias introduced by differing image sizes or pixel ranges.

2.1.2 Denoising

We applied the Non-Local Means denoising algorithm (cv2.fastNlMeansDenoising) to reduce noise while preserving essential sketch contours. This denoising step was critical for removing unwanted noise introduced during image acquisition or scanning, which could interfere with feature extraction. By preserving the important structural information in the sketches, we were able to ensure that the features captured during extraction were representative of the objects in the sketches.

$$I_{denoised} = \text{NonLocalMeans}(I, h = 10, \text{templateWindowSize} = 7, \text{searchWindowSize} = 21) \quad (1)$$

2.1.3 Contrast Enhancement

Histogram equalization was applied to enhance the contrast of the sketches, making features more prominent. This step helped to ensure that essential features, such as object boundaries and texture, were not overshadowed by low-contrast areas in the image. By improving the contrast, we enhanced the classifier's ability to distinguish between important features and background noise:

$$I_{enhanced} = \text{histogramEqualization}(I_{denoised}) \quad (2)$$

2.1.4 Sharpening Filter

A sharpening kernel was applied to emphasize edges and fine details in the sketches. The kernel used for sharpening enhanced the high-frequency components, thereby making the structural details in the sketches more distinguishable. This sharpening step improved the clarity of key features in the sketches, which is essential for accurate classification:

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \tag{3}$$

$$I_{filtered} = I_{enhanced} * K \tag{4}$$

2.2 Feature Extraction Methods

We implemented multiple feature extraction techniques to capture different aspects of the sketch images:

2.2.1 Histogram of Oriented Gradients (HOG)

HOG features capture the distribution of gradient orientations in localized portions of an image, making them particularly effective for capturing shape and structure in sketches. We chose the following parameters:

- Orientations: 9
- Pixels per cell: (8, 8)
- Cells per block: (2, 2)
- Block normalization: L2-Hys

These parameters were selected to capture the essential directional information from the images while minimizing computational overhead. The HOG method is particularly useful for recognizing object shapes in sketches.

2.2.2 Local Binary Patterns (LBP)

LBP features capture local texture patterns by comparing each pixel with its neighbors. We used the uniform LBP method with these parameters:

- Radius: 1
- Points: 8
- Method: uniform

The resulting patterns were then summarized into a histogram, creating a compact feature representation of the image's texture. The use of LBP helped capture important micro-textural details in the sketches that were not captured by other methods.

2.2.3 CNN Features

We leveraged transfer learning using a pre-trained ResNet-34 model to extract deep features from the sketch images. This approach allows us to take advantage of powerful representations learned from large-scale image datasets. The extraction steps were as follows:

- Resizing images to 224×224 pixels (ResNet's input size)
- Converting grayscale images to 3-channel by replication
- Normalizing using ImageNet mean and std values
- Using the output of the penultimate layer (512-dimensional vector) as features

This method allowed us to extract high-level features from the sketch images, which provided a more complex and informative representation compared to traditional hand-crafted features.

2.3 Feature Fusion and Normalization

We concatenated the extracted HOG, LBP, and CNN features into a comprehensive feature vector, which provided a holistic representation of the sketches. To ensure that each feature type contributed equally, we applied standard scaling to normalize the features before training:

$$X_{combined} = [X_{HOG}, X_{LBP}, X_{CNN}] \tag{5}$$

$$X_{scaled} = \frac{X_{combined} - \mu}{\sigma} \tag{6}$$

The combination of multiple feature extraction techniques helped us capture both low-level and high-level characteristics of the sketches, improving the overall classification performance.

2.4 Dimensionality Reduction

We applied Principal Component Analysis (PCA) to reduce the dimensionality of the combined feature vector while retaining 95

$$X_{PCA} = \text{PCA}(X_{scaled}, \text{n}_c omponents \text{n}_c omponents_{\text{n}_c omponents_{\text{n}_c omponents = 0.95)(7)}}$$

This step ensured that the dimensionality of the feature vector was reduced without losing significant information, improving both training time and model generalization.

2.5 Classification Algorithms

We evaluated several classification algorithms to determine the best-performing model for sketch classification:

2.5.1 K-Nearest Neighbors (KNN)

A non-parametric method that classifies objects based on the majority class among its k nearest neighbors. We chose k=3 based on preliminary tests and expected it to work well for this task.

2.5.2 Logistic Regression

A linear classifier that models the probability of each class using the logistic function. We set the maximum number of iterations to 1000 to ensure the model converged.

2.5.3 Support Vector Machine (SVM)

A powerful classifier that finds the optimal hyperplane that maximizes the margin between different classes. We used the Radial Basis Function (RBF) kernel, which is particularly effective for non-linear decision boundaries.

2.5.4 Naive Bayes

A probabilistic classifier based on Bayes' theorem with the assumption of feature independence. We used the Gaussian Naive Bayes variant, which assumes the features follow a Gaussian distribution.

3 Experiments and Results

3.1 Dataset Description

The dataset consists of hand-drawn sketches in PNG format, organized into multiple categories. Each sketch represents an object or concept (e.g., apple, airplane, house). The images are grayscale and contain simple line drawings.

3.2 Experimental Setup

- Training-Testing Split: 80% training, 20% testing (random split with seed 42)
- Feature Dimensionality:
  - HOG features: Varies based on image size and HOG parameters
  - LBP features: 10 (histogram bins)
  - CNN features: 512 (from ResNet-34)
  - Combined features: Sum of all feature dimensions
- Evaluation Metric: Accuracy and classification report (precision, recall, F1-score)

3.3 Results and Analysis

3.3.1 Classification Performance

The following table summarizes the accuracy achieved by different classification algorithms on the test set:

Classifier	Accuracy Without PCA (%)	Accuracy With PCA (%)
KNN (k=3)	26.85	26.67
Logistic Regression	47.675	45.70
SVM (RBF kernel)	43.2	43.2
Random Forest	32.175	27.025
Naive Bayes	31.85	34.65
MLP	42.175	40.175

Table 1: Classification accuracy for different algorithms with and without PCA.

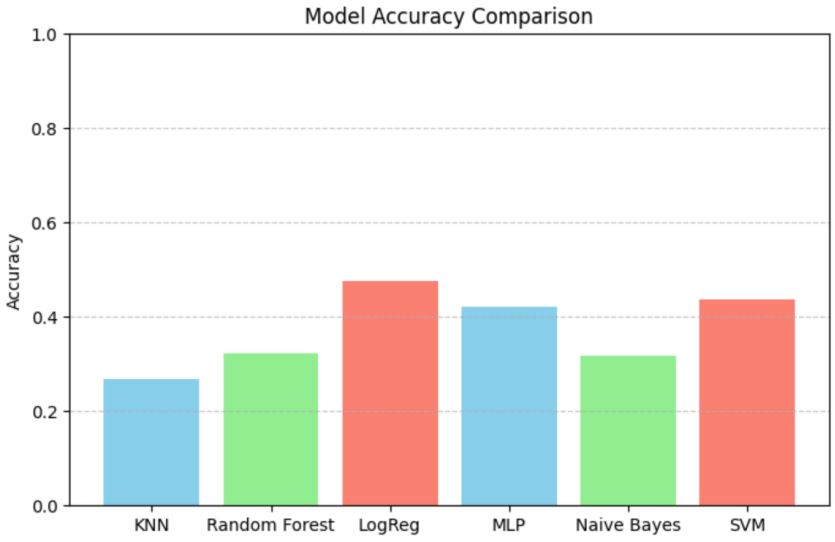

Figure 1: Model Accuracy Comparison

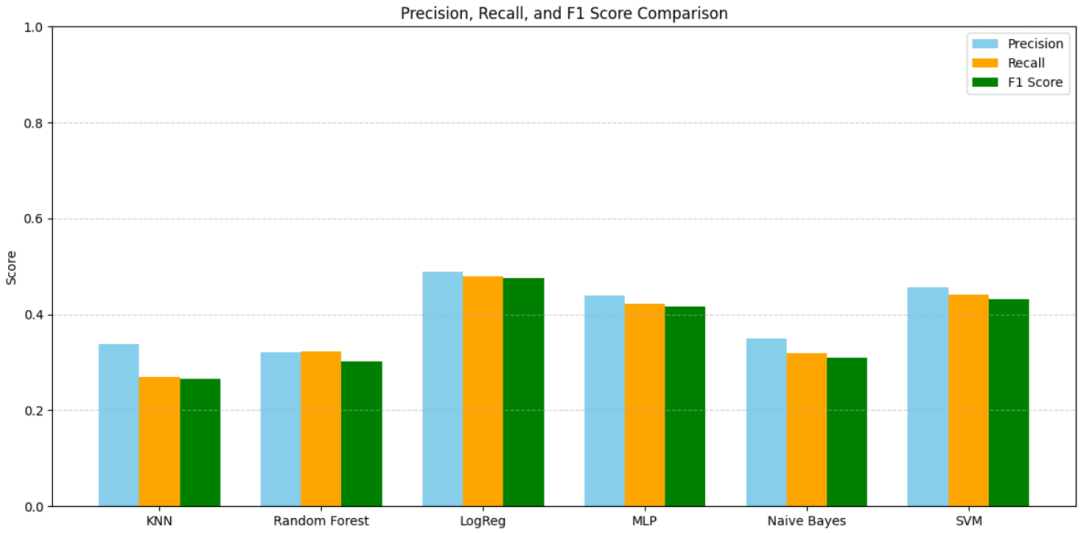

Figure 2: Precision, Recall, and F1 Score Comparison

4 Project Summary

This project focuses on classifying human-drawn sketches into predefined object categories using a combination of traditional and deep learning-based approaches. The team of four students (Pragati Rokade, Vandita Gupta, Jigyasa Tiwari, and Pragati Khandelwal) has developed a comprehensive framework that consists of three key stages:

4.1 Key Components

1. **Image Preprocessing**:
   - Resizing images to 128×128 pixels and normalizing to [0,1] range
   - Applying Non-Local Means denoising to reduce noise while preserving contours
   - Using histogram equalization for contrast enhancement
   - Implementing sharpening filters to emphasize edges and details
2. **Feature Extraction**:
   - Histogram of Oriented Gradients (HOG) to capture shape and structure
   - Local Binary Patterns (LBP) to extract texture information
   - Convolutional Neural Network (CNN) features using a pre-trained ResNet-34
3. **Classification Algorithms**:
   - K-Nearest Neighbors (KNN)
   - Logistic Regression
   - Support Vector Machine (SVM) with RBF kernel
   - Random Forest
   - Naive Bayes
   - Multi-Layer Perceptron (MLP)

4.2 Results

The team evaluated all classifiers both with and without Principal Component Analysis (PCA):

- Logistic Regression performed best with 47.68% accuracy (without PCA) and 45.70% (with PCA)
- SVM achieved 43.2% accuracy consistently

- MLP showed good performance with 42.18% accuracy (without PCA)
- KNN, Random Forest, and Naive Bayes performed less effectively

References

[1] Stanford CS231n, *Image Classification and Recognition*
https://cs231n.stanford.edu/reports/2017/pdfs/420.pdf

[2] *Analysis and Recognition of Hand-Drawn Images with Effective Data Handling*
https://www.researchgate.net/publication/337922291_Analysis_and_Recognition_of_Hand-Drawn_Images_with_Effective_Data_Handling

[3] *Support Vector Machine Algorithm - GeeksforGeeks*
https://www.geeksforgeeks.org/support-vector-machine-algorithm/

[4] IBM, *Support Vector Machine*
https://www.ibm.com/think/topics/support-vector-machine

[5] *Understanding Logistic Regression - GeeksforGeeks*
https://www.geeksforgeeks.org/understanding-logistic-regression/

[6] M. Author, *The Complete Guide to Image Preprocessing Techniques in Python*
https://rb.gy/lqen0r

[7] Analytics Vidhya, *Getting Started with Image Processing Using OpenCV*
https://shorturl.at/K31DW

A Contribution of Each Member

1. **Pragati Rokade (B23CM1055):** Implemented the data preprocessing pipeline and worked on the frontend development.
2. **Vandita Gupta (B23CM1061):** Implemented SVM, MLP, Random forest and also tried to make ensemble of these. Tried PCA technique to enhance accuracies. Worked on report.
3. **Jigyasa Tiwari (B23EE1029):** Implemented feature extraction methods (CNN, HOG ad LBP). Prepared the profile page for the project.
4. **Pragati Khandelwal (B23EE1054):** Coordinated dataset preparation and organization. Implemented Naive Bayes, KNN and logistic regression model. Also prepared the project presentation.